

4장 객체 지향 프로그래밍

I. 객체 지향 프로그래밍과 클래스·객체

1. 객체가 가지는 두 가지 기본 요소로 가장 적절한 것은?

- ① 변수와 상수
- ② 상태와 행동
- ③ 입력과 출력
- ④ 클래스와 패키지

2. 객체 지향 언어의 목적으로 가장 적절한 것은?

- ① 소프트웨어의 재사용을 어렵게 한다.
- ② 프로그램의 실행 속도만 향상한다.
- ③ 소프트웨어 생산성을 높이고 실세계를 쉽게 모델링한다.
- ④ 모든 프로그램을 하나의 함수로 작성한다.

3. 다음 중 객체 지향 프로그래밍의 대표적인 특성이 아닌 것은?

- ① 캡슐화
- ② 상속
- ③ 다형성
- ④ 순차성

4. 데이터와 기능을 하나의 객체로 묶고 외부의 접근을 제한하는 특성은?

- ① 캡슐화
- ② 상속
- ③ 다형성
- ④ 반복

5. 기존 클래스의 속성과 기능을 다른 클래스가 물려받는 것은?

- ① 추상화
- ② 상속
- ③ 캡슐화
- ④ 생성

6. 같은 이름의 메소드가 객체에 따라 다르게 동작할 수 있는 특성은?

- ① 다형성
- ② 캡슐화
- ③ 상속
- ④ 패키지화

7. 복잡한 내부 구현을 숨기고 필요한 핵심 기능만 외부에 보여주는 것은?

- ① 상속
- ② 추상화
- ③ 반복
- ④ 생성

8. 객체를 만들어 내기 위한 설계도 또는 틀을 무엇이라고 하는가?

- ① 메소드
- ② 클래스
- ③ 패키지
- ④ 변수

9. 클래스의 모양대로 생성된 구체적인 실체를 무엇이라고 하는가?

- ① 객체
- ② 메소드
- ③ 필드
- ④ 생성자

10. 객체의 속성(state)을 저장하는 클래스의 멤버는?

- ① 생성자
- ② 필드
- ③ 패키지
- ④ 클래스 이름

II. 클래스 구성과 객체 생성

11. 객체의 행동(behavior)을 구현하는 클래스의 멤버는?

- ① 필드
- ② 메소드
- ③ 패키지
- ④ 레퍼런스

12. 자바에서 클래스를 선언할 때 사용하는 키워드는?

- ① object
- ② new
- ③ class
- ④ create

13. 다음 중 Circle 클래스의 올바른 선언은?

- ① `Circle class { }`
- ② `class Circle { }`
- ③ `new Circle { }`

④ `object Circle { }`

14. 다음 중 객체를 생성할 때 사용하는 키워드는?

- ① `class`
- ② `static`
- ③ `new`
- ④ `this`

15. 다음 코드에서 `pizza`의 역할은?

```
Circle pizza;
```

- ① `Circle` 객체의 레퍼런스 변수
- ② 생성자
- ③ 메소드
- ④ 정수 변수

16. 다음 중 `Circle` 객체를 올바르게 생성한 것은?

- ① `Circle pizza = Circle();`
- ② `Circle pizza = new Circle();`
- ③ `new pizza = Circle();`
- ④ `Circle new pizza();`

17. 객체의 멤버에 접근할 때 사용하는 연산자는?

- ① `+`
- ② `:`
- ③ `.`
- ④ `#`

18. 다음 코드에서 객체의 필드에 값을 저장하는 문장은?

```
Circle pizza = new Circle();  
pizza.radius = 10;
```

- ① 첫 번째 문장만
- ② 두 번째 문장만
- ③ 두 문장 모두
- ④ 두 문장 모두 아님

19. 다음 코드에서 호출되는 메소드는?

```
double area = pizza.getArea();
```

- ① `pizza()`
- ② `getArea()`
- ③ `area()`
- ④ `Circle()`

20. 다음 코드의 실행 결과는?

```
class Rectangle {
    int width = 4;
    int height = 5;

    int getArea() {
        return width * height;
    }
}

Rectangle rect = new Rectangle();
System.out.println(rect.getArea());
```

- ① 4
- ② 5
- ③ 9
- ④ 20

III. 생성자

21. 생성자의 주된 목적은?

- ① 프로그램 종료
- ② 객체 초기화
- ③ 반복문 실행
- ④ 패키지 생성

22. 생성자의 이름은 무엇과 같아야 하는가?

- ① 필드 이름
- ② 클래스 이름
- ③ 패키지 이름
- ④ main 메소드 이름

23. 생성자가 호출되는 시점은?

- ① 클래스를 작성할 때
- ② 객체를 생성할 때
- ③ 프로그램을 종료할 때
- ④ 메소드를 선언할 때

24. 생성자의 리턴 타입에 대한 설명으로 올바른 것은?

- ① 반드시 void이다.
- ② 반드시 int이다.
- ③ 리턴 타입을 지정하지 않는다.
- ④ 클래스에 따라 반드시 달라진다.

25. 한 클래스에 여러 개의 생성자를 작성하는 것은?

- ① 불가능하다.
- ② 하나만 가능하다.
- ③ 가능하며 생성자 오버로딩이라고 할 수 있다.
- ④ static 생성자만 가능하다.

26. 다음 중 올바른 생성자는?

```
class Book {
```

- ① `void Book() { }`
- ② `Book() { }`
- ③ `int Book() { return 0; }`
- ④ `constructor Book() { }`

27. 기본 생성자(default constructor)에 대한 설명으로 올바른 것은?

- ① 반드시 매개변수가 있다.
- ② 매개변수가 없는 생성자이다.
- ③ 반드시 값을 반환한다.
- ④ 객체 생성과 관계없다.

28. 클래스에 생성자가 하나도 선언되지 않은 경우 기본 생성자는 어떻게 되는가?

- ① 실행할 수 없다.
- ② 컴파일러가 자동으로 삽입한다.
- ③ JVM이 클래스를 삭제한다.
- ④ main()이 생성한다.

29. 개발자가 매개변수가 있는 생성자를 하나 작성한 경우 기본 생성자는?

- ① 항상 자동 삽입된다.
- ② 자동 삽입되지 않는다.
- ③ 두 개가 자동 생성된다.
- ④ static으로 변경된다.

30. 다음 코드 실행 후 `b.author` 의 값은?

```
class Book {
    String title;
    String author;
```

```

    Book(String t) {
        title = t;
        author = "작자미상";
    }
}

Book b = new Book("춘향전");
System.out.println(b.author);

```

- ① 춘향전
- ② Book
- ③ 작자미상
- ④ null

IV. 객체 배열과 메소드

31. Circle 객체의 레퍼런스를 저장하는 배열 선언으로 올바른 것은?

- ① `Circle c;`
- ② `Circle[] c;`
- ③ `int[] Circle;`
- ④ `object Circle[];`

32. 다음 코드 실행 직후 실제 Circle 객체의 개수는?

```
Circle[] c = new Circle[5];
```

- ① 0개
- ② 1개
- ③ 5개
- ④ 10개

33. 다음 문장의 기능으로 올바른 것은?

```
c[0] = new Circle();
```

- ① 배열을 삭제한다.
- ② 첫 번째 원소가 Circle 객체를 참조하게 한다.
- ③ 배열 크기를 증가시킨다.
- ④ Circle 클래스를 선언한다.

34. 다음 코드에서 생성되는 Circle 객체의 수는?

```
Circle[] c = new Circle[5];

for (int i = 0; i < c.length; i++) {
```

```
c[i] = new Circle();  
}
```

- ① 1개
- ② 4개
- ③ 5개
- ④ 6개

35. 자바의 모든 메소드는 어디에 있어야 하는가?

- ① 반드시 클래스 안
- ② 반드시 main() 밖
- ③ 반드시 패키지 밖
- ④ 반드시 생성자 안

36. 메소드의 리턴 타입은 무엇을 나타내는가?

- ① 메소드 이름
- ② 반환하는 값의 데이터 타입
- ③ 매개변수 개수
- ④ 클래스 이름

37. 기본 타입의 값을 메소드의 인자로 전달하면 어떻게 되는가?

- ① 객체 전체가 전달된다.
- ② 값이 복사되어 전달된다.
- ③ 변수 자체가 이동한다.
- ④ 원래 변수는 삭제된다.

38. 기본 타입의 값을 전달받은 매개변수를 메소드 내부에서 변경하면 호출한 실인자의 값은?

- ① 항상 변경된다.
- ② 변경되지 않는다.
- ③ 0이 된다.
- ④ null이 된다.

39. 객체를 메소드의 인자로 전달할 때 전달되는 것은?

- ① 객체의 레퍼런스
- ② 클래스 이름
- ③ 객체 전체의 복사본만
- ④ 생성자

40. 배열을 메소드에 전달할 때 올바른 설명은?

- ① 배열의 레퍼런스가 전달된다.
- ② 배열을 전달할 수 없다.

- ③ 배열 길이만 전달된다.
- ④ 배열이 자동 삭제된다.

V. 메소드 오버로딩과 가비지 컬렉션

41. 메소드 오버로딩의 조건으로 올바른 것은?

- ① 메소드 이름이 반드시 달라야 한다.
- ② 같은 이름의 메소드에서 매개변수의 개수나 타입이 달라야 한다.
- ③ 리턴 타입만 달라야 한다.
- ④ 클래스 이름이 달라야 한다.

42. 메소드 오버로딩을 판단할 때 고려하지 않는 것은?

- ① 메소드 이름
- ② 매개변수 개수
- ③ 매개변수 타입
- ④ 리턴 타입

43. 다음 중 올바른 메소드 오버로딩은?

①

```
int sum(int a, int b)
int sum(int a, int b)
```

②

```
int sum(int a, int b)
int sum(int a, int b, int c)
```

③

```
int sum(int a, int b)
double sum(int a, int b)
```

④

```
int sum(int a)
int add(int a)
```

44. 다음 두 메소드는 오버로딩에 성공하는가?

```
int getSum(int a, int b)
double getSum(int a, int b)
```

- ① 성공한다.
- ② 실패한다.
- ③ 생성자가 된다.
- ④ static일 때만 성공한다.

45. 자신을 가리키는 레퍼런스가 하나도 없는 객체를 무엇이라고 하는가?

- ① 클래스
- ② 가비지
- ③ 생성자
- ④ 패키지

46. 자바에서 가비지를 자동으로 회수하는 과정을 무엇이라고 하는가?

- ① 오버로딩
- ② 가비지 컬렉션
- ③ 캡슐화
- ④ 객체 생성

47. 가비지 컬렉션을 수행하는 주체는?

- ① 사용자가 직접 작성한 생성자
- ② 자바 가상 기계의 가비지 컬렉터
- ③ 컴파일러의 main()
- ④ Scanner

48. `System.gc();` 의 의미로 가장 적절한 것은?

- ① 가비지 컬렉션을 반드시 즉시 실행한다.
- ② JVM에 가비지 컬렉션을 요청한다.
- ③ 모든 객체를 즉시 삭제한다.
- ④ 프로그램을 종료한다.

VI. 접근 지정자

49. 자바의 접근 지정자가 아닌 것은?

- ① private
- ② protected
- ③ public
- ④ global

50. 모든 클래스에서 접근할 수 있도록 허용하는 접근 지정자는?

- ① private
- ② public
- ③ protected

④ default

51. 동일한 클래스 내부에서만 접근할 수 있는 멤버의 접근 지정자는?

① public

② protected

③ private

④ default

52. 접근 지정자를 생략한 default 멤버는 기본적으로 어디에서 접근 가능한가?

① 같은 패키지

② 모든 패키지

③ 같은 클래스에서만

④ 어느 곳에서도 접근 불가

53. protected 멤버에 대한 설명으로 올바른 것은?

① 같은 클래스에서만 접근 가능하다.

② 같은 패키지와 상속받은 서브 클래스에서 접근할 수 있다.

③ 모든 클래스에서 항상 접근 가능하다.

④ 어떠한 클래스에서도 접근할 수 없다.

54. 다음 코드에서 다른 클래스가 직접 접근할 수 없는 필드는?

```
class Sample {  
    public int a;  
    private int b;  
    int c;  
}
```

① a

② b

③ c

④ 모두 접근 가능

VII. static과 final

55. static 멤버에 대한 설명으로 올바른 것은?

① 객체마다 각각 생성된다.

② 클래스당 하나만 생성된다.

③ 객체가 생성된 후에만 사용할 수 있다.

④ 다른 객체와 공유할 수 없다.

56. static 멤버를 다른 말로 무엇이라고 하는가?

① 인스턴스 멤버

- ② 클래스 멤버
- ③ 지역 멤버
- ④ 생성자 멤버

57. 다음 중 static 멤버의 특징으로 올바른 것은?

- ① 객체를 생성하지 않고 사용할 수 있다.
- ② 객체마다 독립적으로 생성된다.
- ③ 클래스가 로딩되어도 생성되지 않는다.
- ④ 같은 클래스의 객체가 공유하지 않는다.

58. static 메소드에서 직접 사용할 수 없는 것은?

- ① static 필드
- ② static 메소드
- ③ non-static 멤버
- ④ 클래스 이름

59. static 메소드에서 사용할 수 없는 레퍼런스는?

- ① null
- ② this
- ③ String
- ④ args

60. final에 대한 설명으로 올바른 것은?

- ① final 클래스는 자유롭게 상속할 수 있다.
- ② final 메소드는 자유롭게 오버라이딩할 수 있다.
- ③ final 필드의 값은 실행 중 변경할 수 없다.
- ④ final 필드는 초기값을 가질 수 없다.

4지선다형 정답표

문항	정답	문항	정답	문항	정답
1	②	21	②	41	②
2	③	22	②	42	④
3	④	23	②	43	②
4	①	24	③	44	②
5	②	25	③	45	②
6	①	26	②	46	②
7	②	27	②	47	②
8	②	28	②	48	②
9	①	29	②	49	④
10	②	30	③	50	②

문항	정답	문항	정답	문항	정답
11	②	31	②	51	③
12	③	32	①	52	①
13	②	33	②	53	②
14	③	34	③	54	②
15	①	35	①	55	②
16	②	36	②	56	②
17	③	37	②	57	①
18	②	38	②	58	③
19	②	39	①	59	②
20	④	40	①	60	③

4장 해설

1~10. 객체 지향 프로그래밍과 클래스·객체

1. ② 상태와 행동

객체는 고유한 상태(state)와 행동(behavior)을 가지며 다른 객체와 상호작용합니다.

2. ③ 소프트웨어 생산성을 높이고 실세계를 쉽게 모델링한다

객체 지향 언어는 상속·다형성·캡슐화 등을 이용해 소프트웨어 재사용성을 높이고, 실세계의 객체와 상호작용을 프로그램으로 쉽게 표현하는 것을 목적으로 합니다.

3. ④ 순차성

자료에서 제시한 객체 지향 프로그래밍의 대표적인 특성은 **캡슐화, 상속, 다형성, 추상화**입니다. 순차성은 여기에 해당하지 않습니다.

4. ① 캡슐화

데이터와 기능을 하나의 객체로 묶고 외부에서 함부로 접근하지 못하도록 제한하는 것이 캡슐화입니다.

5. ② 상속

기존 클래스의 속성과 기능을 다른 클래스가 물려받아 재사용하는 것을 상속이라고 합니다.

6. ① 다형성

같은 이름의 메소드나 인터페이스라도 객체에 따라 다르게 동작할 수 있는 특성입니다.

7. ② 추상화

복잡한 내부 구현을 감추고 필요한 핵심 기능만 외부에 보여주는 것이 추상화입니다.

8. ② 클래스

클래스는 객체를 만들어 내기 위한 **설계도 또는 틀**입니다.

9. ① 객체

객체는 클래스의 모양대로 만들어진 실제 존재이며 인스턴스(instance)라고도 합니다.

10. ② 필드

필드는 객체 내부에서 값을 저장하는 **멤버 변수**입니다.

11~20. 클래스 구성과 객체 생성

11. ② 메소드

메소드는 클래스의 멤버 함수이며 객체의 행동을 구현합니다.

12. ③ `class`

자바에서 클래스는 `class` 키워드로 선언합니다.

13. ② `class Circle { }`

`class` 키워드 다음에 클래스 이름을 작성하고 `{ }` 내부에 필드와 메소드를 구현합니다.

14. ③ `new`

객체는 반드시 `new` 키워드를 이용하여 생성하며, `new` 는 해당 객체의 생성자를 호출합니다.

15. ① Circle 객체의 레퍼런스 변수

`Circle pizza;` 는 객체 자체를 생성한 것이 아니라 Circle 객체를 가리킬 수 있는 **레퍼런스 변수**를 선언한 것입니다. 자료에서도 객체 생성의 첫 단계로 레퍼런스 변수 선언을 제시합니다.

16. ② `Circle pizza = new Circle();`

`new Circle()` 로 객체를 생성하고 그 객체의 레퍼런스를 `pizza` 에 저장합니다.

17. ③ `.`

객체의 필드나 메소드는 **점(.) 연산자**를 이용하여 접근합니다.

18. ② 두 번째 문장만

`new Circle()` 은 객체 생성이고 `pizza.radius = 10;` 이 객체의 `radius` 필드에 10을 저장하는 문장입니다.

19. ② `getArea()`

`pizza.getArea()` 에서 `pizza` 는 객체 레퍼런스이고 `getArea()` 가 호출되는 메소드입니다. 자료의 Circle 예제에서도 같은 방식으로 면적 계산 메소드를 호출합니다.

20. ④ 20

`getArea()` 는 `width * height` 를 반환합니다. 따라서 `4 × 5 = 20` 입니다. Rectangle 예제에서도 동일한 방식으로 면적을 계산합니다.

21~30. 생성자

21. ② 객체 초기화

생성자는 객체가 생성될 때 객체를 초기화하기 위해 실행되는 메소드입니다.

22. ② 클래스 이름

생성자의 이름은 반드시 클래스 이름과 동일해야 합니다.

23. ② 객체를 생성할 때

생성자는 `new` 를 통해 객체가 생성될 때 객체당 한 번 호출됩니다.

24. ③ 리턴 타입을 지정하지 않는다

생성자는 일반 메소드와 달리 `void` 를 포함하여 **리턴 타입 자체를 작성하지 않습니다.**

25. ③ 가능하며 생성자 오버로딩이라고 할 수 있다

하나의 클래스에 매개변수가 서로 다른 여러 생성자를 작성할 수 있습니다.

26. ② `Book() { }`

클래스 이름이 `Book` 이면 생성자 이름도 `Book` 이어야 하며 앞에 리턴 타입을 작성하지 않습니다.

27. ② 매개변수가 없는 생성자이다

자료에서는 기본 생성자를 매개변수가 없고 아무 작업 없이 단순 리턴하는 생성자로 설명합니다.

28. ② 컴파일러가 자동으로 삽입한다

개발자가 생성자를 하나도 작성하지 않았다면 컴파일러가 기본 생성자를 자동으로 삽입합니다.

29. ② 자동 삽입되지 않는다

개발자가 생성자를 하나라도 작성하면 컴파일러는 기본 생성자를 자동으로 만들어 주지 않습니다.

30. ③ 작자미상

`new Book("춘향전")` 은 매개변수 하나를 받는 생성자를 호출합니다. 생성자에서 `author = "작자미상"` ; 으로 설정하므로 출력값은 `작자미상` 입니다. 원자료의 `Book` 예제도 동일한 결과를 보여줍니다.

31~40. 객체 배열과 메소드

31. ② `Circle[] c;`

`Circle[]` 은 `Circle` 객체의 레퍼런스를 저장할 배열을 선언하는 형식입니다.

32. ① 0개

`new Circle[5]` 는 `Circle` 객체 5개를 생성하는 것이 아니라 레퍼런스 5개를 저장할 배열을 생성합니다. 각 객체는 별도로 `new Circle()` 을 실행해야 합니다. 자료의 객체 배열 생성 과정에서도 배열 생성 후 각 원소에 객체를 별도로 생성합니다.

33. ② 첫 번째 원소가 `Circle` 객체를 참조하게 한다

`new Circle()` 로 객체를 하나 생성하고 그 레퍼런스를 `c[0]` 에 저장합니다.

34. ③ 5개

반복문이 `i=0` 부터 `i=4` 까지 총 5회 실행되고, 매번 `new Circle()` 을 실행하므로 `Circle` 객체가 5개 생성됩니다.

35. ① 반드시 클래스 안

자바의 모든 메소드는 반드시 클래스 안에 존재해야 합니다.

36. ② 반환하는 값의 데이터 타입

리턴 타입은 메소드가 반환하는 값의 데이터 타입을 의미합니다.

37. ② 값이 복사되어 전달된다

`int` , `double` 등의 기본 타입은 값 자체가 복사되어 매개변수에 전달됩니다.

38. ② 변경되지 않는다

복사된 값을 변경하는 것이므로 호출한 쪽의 원래 실인자 값은 변경되지 않습니다.

39. ① 객체의 레퍼런스

객체 전체가 복사되는 것이 아니라 객체를 가리키는 레퍼런스가 전달됩니다. 따라서 매개변수와 실인자가 같은 객체를 공유하게 됩니다.

40. ① 배열의 레퍼런스가 전달된다

배열도 객체와 마찬가지로 배열의 레퍼런스가 전달됩니다. 따라서 메소드에서 배열의 원소를 변경하면 원래 배열에도 반영될 수 있습니다.

41~48. 오버로딩과 가비지 컬렉션

41. ② 같은 이름의 메소드에서 매개변수의 개수나 타입이 달라야 한다

오버로딩은 메소드 이름은 같고 매개변수의 개수 또는 타입이 달라야 합니다.

42. ④ 리턴 타입

리턴 타입은 오버로딩 성공 여부를 판단하는 기준이 아닙니다.

43. ②

`sum(int, int)` 와 `sum(int, int, int)` 는 메소드 이름은 같지만 매개변수 개수가 다르므로 정상적인 오버로딩입니다.

44. ② 실패한다

두 메소드의 이름과 매개변수 타입·개수가 모두 같습니다. 리턴 타입만 `int` 와 `double` 로 다르므로 오버로딩이 성립하지 않습니다.

45. ② 가비지

자신을 가리키는 레퍼런스가 하나도 없어 프로그램에서 더 이상 사용할 수 없는 객체를 가비지라고 합니다.

46. ② 가비지 컬렉션

사용할 수 없게 된 가비지를 자동으로 회수하여 가용 메모리로 반환하는 과정입니다.

47. ② 자바 가상 기계의 가비지 컬렉터

객체 소멸과 가비지 회수는 JVM의 가비지 컬렉터가 담당합니다.

48. ② JVM에 가비지 컬렉션을 요청한다

`System.gc()` 는 JVM에 가비지 컬렉션을 요청하지만 즉시 실행된다고 보장하지 않습니다. 실제 실행 시점은 JVM이 판단합니다.

49~54. 접근 지정자

49. ④ `global`

자료에서 제시한 자바의 접근 지정자는 `private` , `protected` , `public` , 그리고 접근 지정자를 생략한 `default` 의 네 가지입니다. `global` 은 해당하지 않습니다.

50. ② `public`

`public` 멤버는 패키지에 관계없이 모든 클래스에서 접근할 수 있습니다.

51. ③ `private`

`private` 멤버는 동일 클래스 내부에서만 접근할 수 있으며 서브 클래스에서도 직접 접근할 수 없습니다.

52. ① 같은 패키지

접근 지정자를 생략한 `default` 멤버는 같은 패키지에 속한 클래스에서 접근할 수 있습니다.

53. ② 같은 패키지와 상속받은 서브 클래스에서 접근할 수 있다

`protected` 는 같은 패키지에서 접근할 수 있고, 다른 패키지라도 상속받은 서브 클래스에서 접근할 수 있습니다.

54. ② `b`

`b`는 `private int b;`로 선언되어 있으므로 다른 클래스에서 직접 접근할 수 없습니다. 자료의 `Sample` 예제에서도 `aClass.b = 10;`이 컴파일 오류가 됩니다.

55~60. static과 final

55. ② 클래스당 하나만 생성된다

static 멤버는 객체마다 생성되지 않고 클래스당 하나만 생성됩니다.

56. ② 클래스 멤버

static 멤버는 **클래스 멤버**라고도 부릅니다. 반대로 non-static 멤버는 인스턴스 멤버라고 합니다.

57. ① 객체를 생성하지 않고 사용할 수 있다

static 멤버는 클래스가 로딩될 때 생성되므로 객체 생성 없이 `클래스이름.멤버` 방식으로 사용할 수 있습니다.

58. ③ non-static 멤버

static 메소드는 객체가 없는 상태에서도 실행될 수 있으므로 특정 객체에 속한 non-static 필드나 메소드를 직접 사용할 수 없습니다.

59. ② `this`

`this`는 현재 객체를 가리키는 레퍼런스입니다. static 메소드는 특정 객체에 속하지 않으므로 `this`를 사용할 수 없습니다.

60. ③ final 필드의 값은 실행 중 변경할 수 없다

`final` 필드는 상수를 선언할 때 사용하며 초기화한 이후 값을 변경할 수 없습니다. 또한 `final` 클래스는 상속할 수 없고 `final` 메소드는 오버라이딩할 수 없습니다.